

VokVision: Interim Project Report

Monocular Multi-view 3D Reconstruction for Mobile Users

Team Name Omitted for Blind Review

April 17, 2026

Abstract

This interim report presents the current status of **VokVision**, a mobile-first computer vision system for 3D scene reconstruction from user-captured multi-view RGB images. The current branch (*SAM_implementation*) supports upload, asynchronous job orchestration, MAST3R-based mapping, and Gaussian Splatting training interfaces. We summarize problem scope, user interface progress, related work, dataset and metric choices, preliminary system analysis, compute requirements, individual ownership, and next milestones.

1 Problem Statement

Goal. Build an application where a user captures a short burst of photos of an object or small scene on a phone and obtains a downloadable 3D representation.

Input. The current system accepts a set of monocular RGB images (JPEG/PNG) captured from multiple viewpoints. In the present repository snapshot, our test corpus contains 21 JPEG images.

Output. The intended output is a reconstructed 3D artifact (point cloud / Gaussian representation), delivered as `point_cloud.ply` through a backend download endpoint.

Primary users. Content creators, students, and prototyping teams who need a simple “capture-to-3D” workflow without specialized sensors.

System interface. Users interact through a Flutter mobile app for capture/upload and polling, while a FastAPI backend executes reconstruction asynchronously.

Current scope constraints.

- Monocular image-only pipeline (no IMU/LiDAR fusion yet).
- Reconstruction quality depends strongly on viewpoint coverage and image quality.
- The branch currently does not expose an active SAM segmentation stage in production flow.

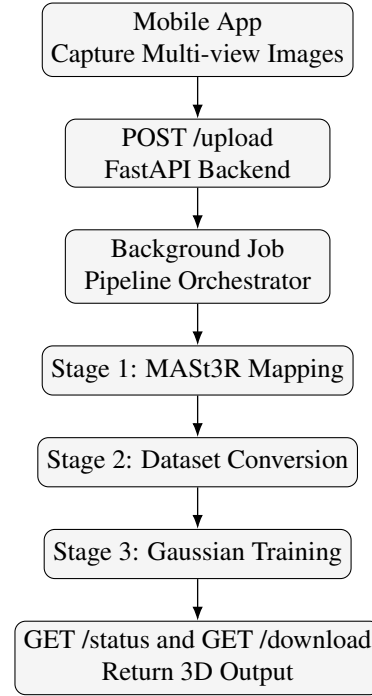


Figure 1: Current end-to-end system design and UI-driven backend workflow.

2 User Interface and Progress

2.1 Interface design

The UI follows a feature-modular Flutter architecture with dedicated modules for authentication, capture, reconstruction, gallery, and editor flows. The backend exposes REST endpoints: root health, upload, status, and download.

2.2 Progress status

Completed components

- Flutter module scaffolding for capture/reconstruction workflow.
- FastAPI upload/status/download API endpoints.

- Background orchestration shell for 3-stage pipeline.
- MAST3R and Gaussian-Splatting repositories integrated in workspace.

Incomplete / in-progress components

- Stage-2 conversion integration mismatch (script contract issue).
- End-to-end success telemetry and error-state reporting.
- Active SAM segmentation stage in deployed branch flow.

3 Related Work

MASt3R / DUS3R family. DUS3R and MAST3R reformulate image matching and geometry estimation directly in 3D, reducing brittle feature engineering and improving reconstruction robustness in many unconstrained scenes [3, 6].

COLMAP-style SfM baselines. Classical SfM systems such as COLMAP remain strong geometry baselines for sparse reconstruction and pose estimation [5].

Neural rendering baselines. 3D Gaussian Splatting (3DGS) enables real-time rendering while maintaining high fidelity, making it practical for interactive applications compared to slower NeRF-style rendering [1, 4].

Segmentation-oriented methods. Segment Anything (SAM) provides promptable segmentation useful for foreground isolation and object-aware reconstruction filtering [2]. This is part of our intended branch roadmap.

Chosen baselines for comparison.

- Classical SfM pipeline: COLMAP [5]
- Neural rendering: NeRF [4]
- Real-time reconstruction target: 3DGS [1]

4 Datasets and Evaluation Metrics

4.1 Dataset snapshot and EDA

Current internal dataset snapshot used for development testing:

- Upload corpus: 22 JPEG images in the latest successful local run.
- Reconstruction workspace assets: image and sparse geometry folders available (`gs_dataset/images`, `gs_dataset/sparse`).

Observed data peculiarities.

- Small-scale internal dataset; not yet diverse across scenes/devices.

Table 1: Current development dataset snapshot from repository workspace.

Asset	Count	Notes
Uploaded images (JPEG)	22	Latest local run batch
<code>gs_dataset/images</code> files	22	Candidate training views
<code>gs_dataset/sparse</code> files	5	Sparse geometry metadata

- Potential viewpoint imbalance (insufficient full 360 coverage).
- Real-world blur/exposure variation expected from handheld capture.

4.2 Evaluation metrics

Module-wise metrics selected for final evaluation:

- **Reconstruction quality:** PSNR, SSIM, LPIPS (rendered view vs. held-out images).
- **Geometry quality:** Chamfer distance / point-cloud completeness where GT exists.
- **System metrics:** end-to-end runtime, GPU memory, CPU RAM, storage footprint.
- **Product metrics:** successful job rate, average time-to-result, API error rate.

Resource relevance. The application is compute-heavy at training stage; GPU VRAM and storage throughput dominate runtime.

5 Analysis of Results (Interim)

At this checkpoint, we report module-level status using the latest successful local execution (`run_local.py --iterations 2500`) on a 22-image project. The pipeline reached end-to-end completion and produced a final `model.ply` artifact.

Latest run highlights (local, CUDA):

- MAST3R reconstruction completed with 2,181,208 points written ($\text{conf} \geq 1.5$).
- Object isolation completed with automatic fallback (multi-scale GrabCut+HSV) when SAM2 module was unavailable.

Table 2: Interim module-wise performance and status (latest local run).

Module	Current result	Key metric/status
Local job orchestration	Functional	22 images, 2500 iterations
MASt3R Stage	Completed	2,181,208 points written
Object isolation stage	Completed	Final cleaned cloud: 8,294 points
OpenSplat training	Completed	2,500 steps, final model written
Output artifact	Generated	<code>model.ply</code> (934.2 KB)

- Multi-cue isolation kept 355,770 points at threshold 0.65 and 317,197 points at threshold 0.70; final cleaned object cloud: 8,294 points.
- OpenSplat completed 2,500 training steps and wrote `model.ply` (934.2 KB) and `cameras.json`.

Observed warnings and operational gaps:

- VLM path was skipped because `GEMINI_API_KEY` was not set.
- SAM2 module was unavailable in environment; fallback segmentation was used.
- HF Hub requests were unauthenticated (no `HF_TOKEN`), with potential rate-limit/performance impact.
- Environment health warnings: Python 3.10 support horizon and deprecated `google.generativeai` package usage.
- MASt3R warning indicated missing CUDA-compiled RoPE2D extension (slow PyTorch fallback).

Bandwidth and storage considerations. Multi-image uploads are manageable on Wi-Fi, but model artifacts can grow quickly; storage lifecycle policies are needed before scale-out.

6 Compute Requirements

Current estimate.

- GPU: NVIDIA CUDA-capable GPU (preferably at least 12GB VRAM; higher is preferred for stability).
- CPU RAM: 16–32GB recommended for preprocessing and parallel services.

- Storage: Fast SSD required for image I/O and output artifacts.

Can we manage current compute requirements? For prototype-scale experiments, yes. For sustained team workloads and repeated training runs, we need either queue-based job scheduling on shared GPU servers or cloud GPU instances.

Adjustments needed.

- Implement resource-aware scheduling (avoid concurrent heavy jobs on one GPU).
- Add output retention/cleanup policy to control storage growth.
- Add fallback execution profiles (reduced image resolution / fewer iterations).
- Migrate from `google.generativeai` to `google.genai` and standardize Python 3.11+ runtime.
- Add environment preflight checks for keys/modules (`GEMINI_API_KEY`, `HF_TOKEN`, SAM2 availability).

7 Next Steps

1. Promote the successful local path into API-triggered production workflow with identical configs (Owner: Member C, Member B).
2. Add explicit job states (queued, running, failed, succeeded) and error payload API (Owner: Member B).
3. Establish reproducible benchmark sheet (PSNR/SSIM/LPIPS, runtime, memory) across multiple scenes (Owner: Member D).
4. Expand dataset diversity and perform controlled EDA (Owner: Member A, Member D).
5. Integrate SAM2 path where available and compare against current GrabCut+HSV fallback baseline (Owner: Member C, Member D).
6. Harden environment/dependency setup (keys, tokens, package/runtime versions) via preflight script (Owner: Member B).

Risk to timeline. The highest risk has shifted from stage integration to environment consistency (API keys, optional modules, package/runtime versions). Mitigation is a strict preflight check and reproducible environment manifest.

References

- [1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuhler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. In *ACM Transactions on Graphics (SIGGRAPH)*, 2023.
- [2] Alexander Kirillov, Eric Mintun, Nikhila Ravi, et al. Segment anything. In *International Conference on Computer Vision (ICCV)*, 2023.
- [3] Martin Laba, Yiming Wang, et al. Mast3r: Grounding image matching in 3d. *arXiv preprint arXiv:2406.09756*, 2024.
- [4] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 2021.
- [5] Johannes L. Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [6] Shuzhe Wang, César Ducor, Yiming Wang, et al. Dust3r: Geometric 3d vision made easy. *arXiv preprint arXiv:2312.14132*, 2023.